Temelash Card — Phased Build Plan

Oct 31, 2026 · @Lidiya Fitsum

1. Read this first

This document turns the finished Temelash Card design prototype into a production build in ten phases, with three release gates: a private pilot after Phase 4, paid launch after Phase 6, and full launch after Phase 9. It is written for a building Claude (or developer) who has the prototype attached but no other context.

What the prototype is. A published Claude Design canvas of 336 clickable boards: 257 for the customer app, staff app and owner app (each in English, Amharic and Afaan Oromoo where it matters), and 79 for the internal Admin back-office. Every screen, message, error state and business rule in the product was designed and click-tested there. Nothing is connected to a server: data is sample data held in the page.

How to use it.

- The prototype is the specification for screens, wording, states and flows. Match it. Where it is silent, ask before inventing.
- Board names are the index. They follow a pattern: `Own…` owner app, `Mgr` branch manager, `Staff…` staff app, `Admin…` back-office, no prefix for the customer app, `Amharic…` / `Oromo…` language variants. Each phase below lists the boards it covers.
- Business rules live in section 2 and in the Admin Settings (Rules and prices) board. Prices, commission, trial length and limits are settings, never hard-coded.
- Section 9 lists where the prototype cheats. Do not copy those shortcuts into production.

Ground rules for the building Claude.

1. The server decides everything that matters: who may do what, what a stamp is, what a payment is. The apps only ask.
2. Money and stamps are append-only records. Corrections are new records that point at the old one. Nothing is overwritten or deleted.
3. Every action by staff, owners and admins writes to an audit log with who, what, when and why.

4. Every user-facing string exists in English, Amharic and Afaan Oromoo from the day the screen is built. Amharic and Oromo text from the prototype is a draft and needs native-speaker review before launch.
5. Dates are stored in UTC and shown in East Africa Time, with the Ethiopian calendar and Ethiopian local-time clock available as display options.
6. Finish a phase completely (acceptance tests passing, exit gate met) before starting the next, except where section 6 allows parallel work.

2. The product and its business rules

Temelash Card is a digital stamp-card system for cafés, salons, spas, restaurants and car washes in Ethiopia. A customer scans a shop's QR code once, gets a card, and collects stamps per visit toward rewards. Staff add the stamp by scanning the customer's rotating QR. Owners set the rules and watch results. Temelash (Modern Ethiopia Marketing PLC for now) sells yearly plans to the businesses, sometimes through sales agents who earn commission.

Product decisions already made, which the build must respect:

- Stamp-based loyalty only. No coupons or discounts for now.
- Customers sign in with phone number and PIN. No SMS check at customer signup. No idle sign-out.
- Staff accounts are created and handed out by the owner. Staff of every business sign in from one shared `/staff` link, with no business name on the page.
- Staff and owner PINs expire (default 90 days, owner can change). The owner sets an hourly stamp limit per staff member.
- A customer can only be stamped once per visit.
- Customer app languages: English, Amharic, Afaan Oromoo. Tigrinya, Somali, Sidama, Wolaytta, Afar and Gurage show as "Coming soon".
- An owner closes the business by sending a closing request, which Temelash approves; everything keeps working until the approved date. Customers' cards stay safe in their phones.
- Owners order printed QR stands from Temelash inside the app, and can also download QR files.

Money rules. These are the prototype defaults; every number is a setting in Admin, not code.

Rule	Default	Notes
Starter plan	19,000 Birr per year	One branch, core features
Full plan	25,000 Birr per year	Two branches included, all features
Extra branch	5,000 Birr per year	Raises the yearly fee
Free trial	30 days	Then the plan fee is due
Pilot	Free, no charge	Chosen by an admin at approval
Agent commission	Starter 3,000, Full 5,000, extra branch 1,000 Birr	Earned once the business has paid
Government share	30% of each commission	The cost of a commission is commission plus 30%
Payments	Recorded by hand	Telebirr, CBE Birr, bank, cash. No online gateway yet
Finance year	Ethiopian calendar	Year 2019 runs 8 Jul 2026 to 7 Jul 2027 (Hamle 1 to Sene 30)
Next-year payments	Open on Ginbot 1	New year counts from Hamle 1

Agent flow: an admin adds an agent, links each signed-up business to its agent (by the agent's phone number at signup or by an admin "deal"), the agent earns commission when the business pays, and an admin marks commission as paid. The 30% government share is tracked per month and marked as handed over separately.

3. The four apps and who uses them

Four surfaces share one backend. Customer, staff and owner screens are mobile-first (390 wide in the prototype); Admin is a desktop back-office (1280 by 800).

App	Who	Where it runs	What it does	Key prototype boards
Customer	A shop's customers	Telegram Web App (mini app), Next.js	Language picker, sign up or in with phone and PIN, scan a shop QR to join, show a rotating QR to be stamped, see cards, rewards, history, notifications, delete account	Welcome, Main, CouponDetail, ScanJoin, StampEarned, Redeem*, QrFull, Settings, DeleteAccount
Staff	Cashiers and baristas	Web app at the shared `/staff` link	Sign in, set PIN, scan a customer to stamp, type a code by hand when the camera fails, redeem rewards, see recent stamps, work offline	StaffSignIn, StaffScan, StaffFound, StaffAdded, StaffManual, StaffOffline, StaffQueued, StaffRedeem, StaffLimit

App	Who	Where it runs	What it does	Key prototype boards
Owner and branch manager	The business owner; a manager of one branch	Web app, mobile-first	Today, approvals of hand-typed stamps, customers and fixes, team and PINs, card and reward rules, branches and hours, reports, news to customers, posters and stands, plan and payments, inbox, requests to Temelash, security	Own* (owner), OwnMgr* (manager), OwnPlan, OwnInbox, OwnMyRequests, OwnStands
Admin	Temelash staff	Desktop web app at `/administrator`	Businesses, requests inbox, audience, agents and leaderboard, finance and report, messages to owners, stands, activity log, settings, team and roles, safety	All Admin* boards

Roles inside Admin. Owner (everything), Finance (payments, agent payouts, government share, money exports), Support (businesses, requests, messages; no money). The prototype shows these but does not enforce them.

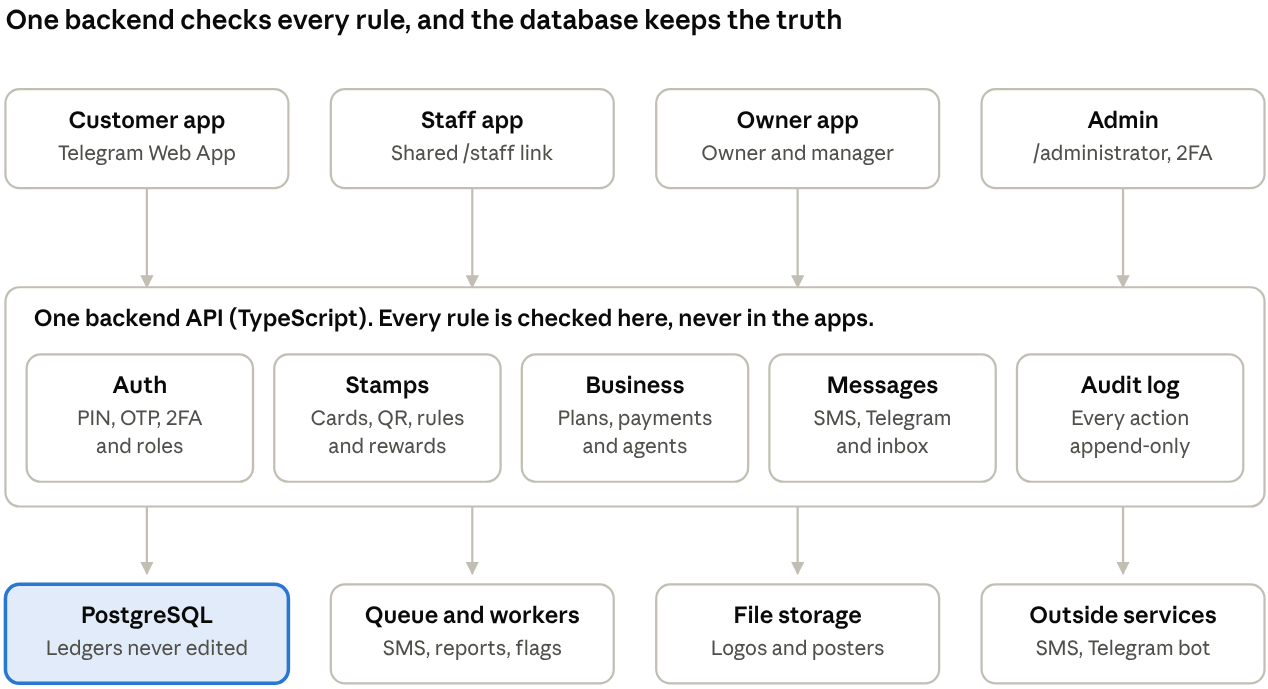
Roles inside the owner app. The owner can do everything. A branch manager is limited to one branch: they see that branch's customers, team and reports and cannot change card rules, plan, money or other branches. The board `OwnMgrOwnerOnly` shows the wall the manager hits.

What each side may never do.

- Customers can only see their own cards and history.
- Staff can only add stamps and redeem at their own branch, within their hourly limit.
- Owners can fix stamps (remove, add, undo, extend, pause a card) but every fix is logged and visible to Temelash.
- Admin can look at what an owner sees, read only, and every look is logged.

4. Recommended architecture

Build one backend that all four apps talk to. The apps hold no business rules: they ask, and the server decides. Start as a single deployable service with clear internal modules (auth, stamps, business and money, messages, audit); split a module out only when load or team size demands it.



architecture · 4 apps, 1 API, data and services

The stack and provider choices, with reasons and the questions to settle with Robel, are in section 11. Three design rules follow from the picture:

- **Stamps and money are rows added, never edited.** The database role used by the app cannot update or delete them.
- **The audit log is written in the same database transaction as the action it records.** No action without a log row, no log row without an action.
- **Slow or flaky work goes through the queue.** SMS, Telegram messages, report builds, audience counts and flag checks run in workers with retries, so a slow provider never slows a stamp.

5. Data model

Start from these entities. Names are suggestions; the shape is what matters. Tables marked append-only get no update or delete path in code, and the database role used by the app has no permission to do either.

Entity	Key fields	Rules
business	name, category, city, status (pending, trial, active, paused, closing, closed, rejected), plan, plan_date, agent_id	Status changes only through a request or an admin action, each logged
branch	business_id, name, address, hours, status	Count drives the yearly fee
owner, manager, staff (one `person` table with `role`)	phone, name, role, branch_id (managers, staff), pin_hash, pin_set_at, locked_until	PIN stored hashed with a slow hash; PIN age drives the 90-day expiry
customer	phone, pin_hash, language, status	One customer, many cards. Deleting an account anonymises, never leaves orphaned stamps
card	customer_id, business_id, status (active, paused, expired, redeemed), reward_ladder_version	One active card per customer per business

Entity	Key fields	Rules
stamp (append-only)	card_id, branch_id, staff_id, at, method (scan, typed), approval (none, waiting, approved, rejected), voided_by	A fix is a new row (void or add) pointing at the old one
reward and redemption (append-only)	card_id, tier, code, issued_at, redeemed_at, redeemed_by, expires_at	A code redeems once
rule set (versioned)	stamps_needed, ladder, once_per_visit, hourly_limit_per_staff, pin_days, hours	A change creates a version; old cards keep the version they started under
request	type (signup, closing, lost phone, deletion, stand order, flag), business_id, stage, assignee, result	Everything an owner asks Temelash for lives here
payment (append-only)	business_id, amount, method, reference, at, finance_year, fixes[]	A correction or reversal is a new row linked to the original; receipt number `TR-yyyymmdd-nn`
plan_fee ledger	business_id, finance_year, branches, fee, paid, balance	Derived from payments, never typed
agent	name, phone, area, status	
deal	agent_id, business_id, plan, commission, status (waiting, paid)	Commission earned only once the business has paid
agent_payout (append-only)	agent_id, amount, at, method, reference	
government_share	month (Ethiopian), amount, handed_over_at, how, reference	One row per month

Entity	Key fields	Rules
message	audience, title, body, sent_at, recipients[], opened_at per recipient	Admin to owners; owners to customers use a separate `news` table with sending limits
stand_product, stand_order, stock_move	name, price, on_sale, stock, warn_at; order stages Placed, Confirmed, Printing, On the way, Delivered, receiving report	Stock changes are rows, not edits
admin_user	email, role, status, totp_secret (encrypted), backup_code_hashes, last_login	Roles Owner, Finance, Support; at least two Owners at all times
audit_log (append-only)	actor_type, actor_id, business_id, type, text, detail, at, ip	Written in the same transaction as the action it records
follow_up, note	business_id, text, due, assignee, done	Internal to Temelash
setting	key, value, changed_by, changed_at	Prices, commission, government percent, limits, thresholds, kill switch

Derived, not stored by hand: balance due, covers-until date, commission owed, government share per month, leaderboard, audience unique-client counts (refreshed every 12 hours), unusual-activity flags.

Time: store UTC. Store the finance year and Ethiopian month on money rows when written, so a report never depends on a later calendar rule change.

6. Roadmap at a glance

Ten phases lead to three releases: a private pilot, a paid launch and a full launch. The pilot comes early on purpose, right after the first four phases, so real shops use the product while money, messages and the rest are still being built.

Pilot after Phase 4 (about 14 to 20 weeks), full launch after Phase 9 (about 26 to 36)

Calendar weeks for a team of 2 to 3 developers. Estimates to re-base after Phase 0.

Phase 0 · Foundations Repo, CI, seed data, translations and calendar library	1 to 2 wks
Phase 1 · Identity and permissions Sign-in for every role, server-side permission checks	2 to 3 wks
Phase 2 · The stamp loop Join, stamp, reward, redeem, offline queue, fixes	4 to 6 wks
Phase 3 · Owner app Rules, team, customers, reports, branch manager	4 to 5 wks
Phase 4 · Admin core and sign-up Login with 2FA, businesses, requests, log, kill switch	3 to 4 wks
Release 1 · Private pilot	5 to 10 friendly shops, free plan, money tracked outside
Phase 5 · Money Payments, receipts, agents, 30% government share, exports	3 to 4 wks
Phase 6 · Messages and news Admin to owners, owners to customers, SMS and Telegram	2 to 3 wks
Release 2 · Paid launch	Money reconciled for a month, agents earning
Phase 7 · Stands Stand shop, orders, production list, stock	2 wks
Phase 8 · Trust and recovery Flags, account recovery, closing, privacy, support tools	2 to 3 wks
Phase 9 · Hardening and languages Native review, load and security tests, restore drill	3 to 4 wks
Release 3 · Full launch	Public signup, three languages reviewed, licence and hosting done

roadmap · 10 phases, 3 release gates

Phases 0 to 4 are strictly in order. After Phase 4, a second developer can build Phase 7 (stands) alongside Phases 5 and 6, since stands touch little else. Phase 3 (owner app) can start in the last two weeks of Phase 2 once the stamp APIs are stable. Phase 8 should not be left until the end in practice: build its flag rules during Phase 4 and finish the rest after paid launch.

7. Phase by phase

Each phase lists its goal, what to build, the prototype boards to follow, how to prove it works, and the gate that lets the next phase start. "Acceptance" items are written to become automated tests.

Phase 0 — Foundations

Goal: a repo and pipeline where the first real feature can ship safely.

- Decide the open items in section 11 with Robel and write the answers into an `ARCHITECTURE.md`.
- Monorepo, environments (local, staging, production), CI that runs lint, type-check and tests on every change, one-command deploy to staging.
- Database with migrations, seed script that loads the prototype's sample businesses, agents and requests so every screen has data.
- Design tokens and shared components taken from the prototype (colours #e42330 red, #4661ad blue, #101827 navy, #fba919 amber; Myriad Pro wordmark; the text-size and contrast rules from the prototype pass).
- Internationalisation skeleton with English, Amharic and Afaan Oromoo files and a check that fails the build when a key is missing in any language.
- Ethiopian calendar and time library with tests (see section 8).
- Error tracking, structured logs, backups, secrets management.

Acceptance: a blank page in each of the four apps deploys to staging from CI; a missing translation key fails the build; the calendar library converts 2026-10-31 to Tikimt 21, 2019 and 2026-07-08 to Hamle 1, 2018 and back.

Exit gate: staging is live, seeded and reproducible from scratch in under 30 minutes.

Phase 1 — Identity, businesses and permissions

Goal: every kind of person can sign in safely, and the server enforces who may do what.

- Customer: phone and PIN, language picker first, five wrong tries lock for 15 minutes, Telegram Web App launch and `initData` verification.
- Owner: sign up with SMS code then PIN; sign in with PIN plus SMS code; forgot-PIN flow.
- Staff: shared `/staff` sign-in, owner-issued first PIN that staff must change, weak-PIN rejection, PIN expiry, lockout, branch picker for staff in several branches.

- Roles and permission checks as server middleware, with one test per row of the permission matrix (owner, manager, staff, customer, admin roles).
- Business, branch and person records; a request record for sign-up.
- Audit log framework used by every later phase.

Boards: Welcome*, ScanJoin*, ScanJoinLock, StaffSignIn*, StaffSetPin*, StaffPickShop, OwnSignIn*, OwnOtp*, OwnForgot*, OwnSignUp*, OwnMgrSignIn, OwnMgrFirstPin.

Acceptance: a manager cannot read another branch by changing an id in the URL; five wrong PINs lock the account and the sixth correct one is still refused until the lock ends; an expired staff PIN forces a change before any stamp can be added.

Exit gate: security review of the auth flows passes; permission tests cover every endpoint.

Phase 2 — The stamp loop (the heart of the product)

Goal: a customer joins a shop, gets stamped, reaches a reward and redeems it, with no owner or admin screens needed yet.

- Join by shop QR or by shop code; one card per shop; the paused, expired and redeemed card states.
- Rotating customer QR that changes every minute; staff scan validates it on the server.
- Stamp rules: once per visit, per-staff hourly limit, wrong shop, no card, duplicate, expired code, near-limit warning.
- Typed-code fallback with the "needs approval" path for hand-typed stamps.
- Reward ladder and redemption: ready, confirm, a one-time code, staff redeem, used and expired states. Rewards that need the network refuse to redeem offline.
- Staff offline queue: stamps taken without network are held on the device, shown as queued, and sent in order when the connection returns, with the server deciding if each is still valid.
- Fixes by owners: remove a stamp, add a stamp, undo, extend, pause or resume a card.
- Customer history, notifications list, stamped-today and stamp-earned moments.

Boards: Main, CouponDetail, StampEarned, StampedToday, History, RedeemReady, RedeemConfirm, RedeemCode*, RedeemDone, StaffScan, StaffFound, StaffAdded, StaffDuplicate, StaffBadCode, StaffManual, StaffCamera, StaffOffline, StaffQueued, StaffRewardNeedsNet, StaffRewardUsed, StaffRewardExpired, StaffRewardDone, StaffLimit, StaffWrongShop, StaffNoCard, StaffPaused, CardPaused, ExpiredCard, RedeemedCard, OwnFix*.

Acceptance: two staff scanning the same customer within one visit produce exactly one stamp; a code used twice redeems once; replaying a queued stamp after its card was paused is rejected with a clear message; the ledger sums to the card's stamp count after every test run.

Exit gate: a load test of 50 stamps per second sustained for 10 minutes with no duplicate or lost stamps; the stamp ledger can be rebuilt from raw rows.

Phase 3 — Owner app

Goal: an owner can run the shop's loyalty programme without calling anyone.

- Today screen with approvals count, alerts, closing banner.
- Approvals and "to check" lists for hand-typed stamps; customers list, search, customer detail, hidden customers.
- Team: add staff and managers, issue and reset PINs, edit hourly limit, unlock locked staff, roles page.
- Card rules designer: stamps needed, reward ladder with warnings, gifts, rules page; changes create a new rule version.
- Shop details, opening hours per day, branches and branch form, look (card colours, logo).
- Posters and QR downloads (tent card, all sizes).
- Reports for 7, 14, 30 days and per branch.
- Activity log, alert settings, security page, date and time settings (including Ethiopian calendar and Ethiopian local time).
- Branch manager view of all the above, limited to one branch.

Boards: OwnToday, OwnApprovals, OwnToCheck, OwnCustomers*, OwnCustomer*, OwnTeam, OwnStaff*, OwnRules, OwnRoles, OwnCard*, OwnGifts, OwnDetails, OwnHours*, OwnBranches, OwnBranchForm, OwnLook, OwnPoster*, OwnReports*, OwnLog, OwnAlertSet, OwnSecurity, OwnDateTime*, OwnHelp, and every OwnMgr* board.

Acceptance: changing the reward ladder does not change the cards already in progress; a branch manager's every request for another branch returns forbidden; reports match a hand count of the ledger for the seeded data.

Exit gate: three real shop owners (not Temelash staff) complete the setup of a new shop unaided in under 20 minutes, observed.

Phase 4 — Admin core, sign-up and the private pilot

Goal: Temelash can approve real businesses and watch them, safely. This is the first release gate.

- Admin login at `/administrator`: email and password, authenticator code (TOTP), backup codes, five tries then a 15-minute lock, sign-out after 30 idle minutes, first-time setup of the authenticator.
- Overview, Businesses list and business page (Overview, People, Customers, Finance, Support, History tabs), search across businesses, agents and requests.
- Requests inbox with the sign-up check (call the owner, match the agent by phone), approve with trial or free pilot, reject with a reason.
- Activity log with filters.
- Settings: rules and prices, team and roles, account and security.
- Kill switch and the basic unusual-activity flags (Phase 8 refines them).
- Sign-up on the owner side connected end to end: sign up, wait for approval, get the message, start stamping.

Boards: AdminLogin*, AdminOverview, AdminBusinesses, AdminBusiness*, AdminRequests, AdminReqSignup*, AdminApproveSignup, AdminAudit, AdminSettings*, AdminSettingsSafety, AdminKillSwitch, AdminCustomer*, AdminPauseBusiness.

Acceptance: an admin with the Support role cannot reach any money screen or money export; the kill switch stops every stamp and redemption within 5 seconds and shows the owner message; every admin action appears in the log with the right actor.

Exit gate: Release 1, private pilot. Five to ten friendly businesses on the free pilot, money tracked outside the system until Phase 5, a written rollback plan, and an on-call person for the first two weeks.

Phase 5 — Money: plans, payments, agents, government share

Goal: every Birr in and out is recorded once, traceable, and reportable.

- Plans and fees per business: Starter, Full, Pilot, extra branches, trial, balance, covers-until date. Owner side: plan page (trial, due) from the same ledger.
- Record a payment (method, reference), receipts with number `TR-yyyymmdd-nn`, printable page, send receipt to the owner's inbox.
- Corrections and reversals as new linked rows; balances and agent commission recalculated from rows.

- Finance year logic on the Ethiopian calendar, finance list, next-year payments opening on Ginbot 1, overdue and due-soon lists.
- Agents: add, deals, link a lead by phone number, commission earned when the business pays, commission owed, "who to pay" list, payout records, payout history, printable statement, leaderboard (7, 14, 30, 60, 90 days).
- Finance report: revenue, commission, 30% government share, left for Temelash, by month and by agent. Government share by month marked handed over with date, how and reference.
- Authenticated exports: activity log, finance report, agent data, businesses. Each export asks again for password and authenticator code, every time, and is logged.
- Money screens and exports limited to the Owner and Finance admin roles.

Boards: AdminFinance*, AdminRecordPay, AdminAddBranch, AdminChangePlan, AdminReceipt*, AdminGovMark, AdminAgents*, AdminAgentPay, AdminAddDeal*, AdminLeadLink, AdminStatement, AdminLeaderboard*, AdminFinanceReport*, AdminExport*, OwnPlan*.

Acceptance: reversing a payment restores the balance and removes the commission the same day; the sum of all payments minus commission minus government share equals "left for Temelash" for every period; an export with a wrong code writes nothing and counts toward the lock.

Exit gate: the finance owner reconciles one month of real pilot payments against the system and the totals match to the Birr.

Phase 6 — Messages, news and notifications

Goal: everyone hears what they need to hear, in their language, through the right channel.

- Admin to owners: compose, audience (every business, payment due, one agent's businesses, one business), templates (payment reminder, next-year payments), optional SMS, history with opened counts, "send again" to those who did not open.
- Owner app inbox with unread count, message page, "my requests" with live status.
- Owner to customers: news and offers with sending limits, preview, sent confirmation. Customer notifications list and Telegram bot messages for stamps, rewards and news.
- SMS provider integration with delivery status, retry and a monthly cost report; templates in all three languages.
- Quiet hours and a per-customer opt-out.

Boards: AdminMessages, AdminMessageCompose, OwnInbox, OwnMessage, OwnMyRequests, OwnNews*, Notifications, AmharicNotifications, OromoNotifications.

Acceptance: a message to "payment due" reaches exactly the businesses with a balance; the opened count updates when the owner reads it; a customer who opted out never receives news.

Exit gate: Release 2, paid launch. Phases 5 and 6 done, a month of reconciled pilot money, support playbook written.

Phase 7 — Stands and printed material

Goal: owners can order printed QR stands and Temelash can produce and deliver them.

- Owner stand shop and cart, order placed, order status page with the five steps, receiving report ("all arrived" or "there was a problem" with a note).
- Admin stands page: production list built from open orders, prices (and hide from owners), stock with low-stock warnings, stock moves.
- Stand requests in the admin inbox with stage movement, replacement or close-without-replacement after a problem, cash received.
- Stand orders and stock changes in the audit log.

Boards: OwnStands, OwnStandCart, OwnStandPlaced, OwnStandOrders, OwnStandOrder*, AdminStands*, AdminReqStand*.

Acceptance: an order moves through all stages and each stage notifies the owner; the production list always equals the open orders; stock never goes below zero.

Exit gate: ten real stand orders delivered and received through the system.

Phase 8 — Trust, safety, recovery and privacy

Goal: the system stays honest and recoverable when people lose phones, make mistakes or misbehave.

- Unusual-activity flags with editable thresholds (stamps per hour on one phone, share typed by hand in a week, new cards in one hour in one shop), flag requests, owner alerts.
- Admin account recovery: lost phone and backup codes, request, another Owner approves with their own code, 24-hour wait, email at every step, new-device alert.

- Owner lost phone and phone change with code check, owner privacy request, customer account deletion (30-day window), business closing flow end to end including the approved closing date and cancel closing.
- Support tools: follow-ups, notes, read-only view as owner (logged), edit details, merge duplicates, close a branch, assign requests to teammates.
- Data export and deletion for customers on request; retention schedule for logs and old stamps.

Boards: AdminReqFlag, AdminSettingsSafety, AdminLoginRecover*, AdminTeamRecovery, AdminRecoveryApprove, AdminBusinessSupport, AdminViewAsOwner, AdminEditBusiness, AdminFollowUp, AdminRequestAssign, AdminReqClosing, AdminReqDeletion, AdminReqLostPhone, OwnCloseBiz*, OwnLostPhone*, OwnChangePhone*, OwnPrivacy*, DeleteAccount.

Acceptance: a script that stamps 100 times in an hour from one phone raises a flag within one minute; a recovery request cannot be approved by the person who made it; merging two shops leaves one set of customers with no duplicate cards.

Exit gate: a tabletop exercise: lose an admin phone, lose an owner phone, flood a shop with fake stamps. Each is handled from the playbook in under one hour.

Phase 9 — Localisation, hardening and full launch

Goal: ready for the public, in three languages, at scale.

- Native-speaker review of every Amharic and Afaan Oromoo string, including layout checks (the prototype already needed fixes for clipped Amharic and Oromo text).
- Accessibility pass: text size and contrast, screen reader labels, tap targets.
- Performance and load tests, cache for reports and audience counts, database indexes, slow-query review.
- Independent security review or penetration test; fix and retest.
- Backup restore drill, disaster recovery plan, uptime monitoring and alerting, status page.
- Hosting and licensing sorted for the Temelash entity; terms and privacy policy published in all three languages.
- Remaining languages kept as "Coming soon" until a translator is lined up.

Acceptance: zero untranslated strings in any shipped language; p95 stamp scan under 1.5 seconds on a mid-range Android phone on a 3G-class connection; restore drill recovers the database to within 15 minutes of loss.

Exit gate: Release 3, full launch. All of the above, plus a signed-off launch checklist from section 10.

8. Requirements that apply to every phase

Security

- Hash PINs and passwords with a slow, salted hash (argon2id or bcrypt). Never log PINs, codes or tokens.
- Admin: TOTP authenticator plus 10 one-time backup codes shown once and kept only as hashes. Authenticator secrets encrypted at rest.
- Sessions: short-lived access token plus rotating refresh token; admin sessions end after 30 idle minutes; sign-out ends every session of that person.
- Rate limit by phone, by IP and by device on every sign-in, code and scan endpoint. Lockout rules come from settings (5 tries, 15 minutes in the prototype).
- Rotating customer QR: signed, expires after one minute, single-use per visit window; the server checks signature, shop and time.
- Re-authenticate (password plus authenticator code) for every export and for the kill switch, and for approving an admin account reset.
- No admin action by an admin can be hidden from the log, including looking at an owner's view.
- Backups encrypted, tested monthly, kept in a different region from the main database.

Audit log. One table, written in the same database transaction as the action. Every entry carries actor, role, business, type, text, detail and time. The prototype's activity-log filters (type, business, person, date) are the minimum query set.

Languages. English, Amharic, Afaan Oromoo, chosen on first launch and changeable in settings. All messages sent by the system (SMS, Telegram, in-app) are templates with a version per language. Allow 30 to 40 percent more width for Amharic and Oromo; the prototype fixed clipping in tabs, cards and headlines and those fixes must be kept.

Dates and times. Store UTC. Show East Africa Time. Owners and customers can choose how dates and times appear, including the Ethiopian calendar and the Ethiopian local-time clock written in brackets after the standard time (for example 14:05 (8:05 LT)). Build and test one library: Gregorian to Ethiopian and back, month names in all three languages, the 13th month of 5 or 6 days, and the finance year from Hamle 1 to Sene 30.

Messaging. SMS through a local Ethiopian provider (to be chosen, see section 11) for owner codes, owner messages and optional customer reminders. Telegram bot for customers who opted in. Every message has a delivery status and a cost record.

Privacy. Collect the least: customer phone, language, cards and stamps. Customers can delete their account (30-day window); owners can ask for a privacy report. Keep stamp and money records after deletion in anonymised form. Publish terms and privacy policy in all three languages before launch.

Offline and weak networks. Staff app must work through short outages (queue and replay), and the owner and customer apps must show a clear offline state rather than a blank page. Keep the first load of each app small enough for a mid-range Android phone on a slow connection.

Performance targets (starting values, adjust after Phase 2 load test). Stamp confirmed in under 1.5 seconds at p95. Owner Today screen under 2 seconds. Admin lists page in under 1 second with 5,000 businesses.

Observability. Structured logs with a request id, error tracking on all four apps, dashboards for stamps per minute, failed sign-ins, SMS failures, queue depth, and unusual-activity flags raised.

9. Prototype shortcuts that must not reach production

The prototype was built to be looked at and clicked, not run. These are the known cheats.

Prototype behaviour	Production replacement
Sample data lives in the page and resets on reload	Real database; the seed script recreates the sample data for staging and demos only
Admin roles (Owner, Finance, Support) are shown but nothing is blocked	Server-side permission check on every endpoint, with a test per role and endpoint
Demo credentials: password `Temelash2026!`, code `123456`, backup code `DEMO-0003`	Real hashing, real TOTP, real backup codes; no demo credentials outside staging
Authenticator code is a fixed number	Real TOTP with a 30-second window and replay protection
Exports are built in the browser	Built on the server, streamed, logged, and again gated by re-authentication

Prototype behaviour	Production replacement
Admin messages and stand prices are separate from what the owner app shows	One message table and one stand catalogue feed both apps
"Today" is fixed to 30 Oct 2026	Real clock; all due, late and overdue logic reads server time
The kill switch only changes a banner	A server flag checked inside the stamp and redeem code paths
Unusual-activity flags are computed from sample counts	Rules run against the stamp ledger on a schedule and on each stamp
Receipts print from the browser	Receipt page and PDF generated from stored rows; ask the accountant whether tax or fiscal details are required
Amharic and Afaan Oromoo text is draft	Native-speaker review, then locked in translation files
CSV phone numbers starting with "+" get a leading apostrophe so Excel does not treat them as formulas	Keep the protection; document it for finance staff
Photos, logos and posters use placeholders	Real upload, storage, resize and virus scan
Telegram, SMS, Telebirr and QR camera are simulated	Real integrations, each behind an interface so a provider can be swapped

Known gaps not designed yet. Online payments (Telebirr or Chapa) for plan fees; a "Partner" plan (shown as coming soon); languages beyond the first three; a public marketing site and signup landing page; a customer-facing web fallback outside Telegram; push notifications for the owner app; automatic invoice generation; multi-currency; reseller or franchise accounts.

10. Testing, pilot and launch

Testing layers. Unit tests for rules (stamp, limit, ladder, commission, calendar). API tests for every endpoint and role. End-to-end tests that replay the prototype's click-through flows (the prototype test scripts for the admin covered 150 checks and are a good list of flows to reproduce). Load test in Phase 2 and again in Phase 9. Language test: every screen opened in all three languages at a narrow width.

Pilot (Release 1).

1. Choose 5 to 10 friendly businesses across café, salon and car wash, in at least two parts of Addis Ababa.
2. Free pilot plan; owners told in person what is and is not final.
3. A named person from Temelash visits each shop in the first week and watches real staff stamp real customers.
4. Daily check for two weeks: stamps per shop, failed scans, flags, support requests, SMS failures.
5. Weekly review with Robel; a defect list ranked by how often it hit real people.

Paid launch (Release 2). Pilot shops move to a paid plan or leave; agents start earning commission; finance reconciles month one; support hours and escalation named.

Full launch (Release 3). Public signup, agents recruited across regions, printed stands in production, all three languages reviewed.

Launch checklist.

- [ ] Security review findings closed
- [ ] Backup restore drill passed in the last 30 days
- [ ] On-call rota for the first month
- [ ] Kill switch tested on production with a test shop
- [ ] Terms and privacy policy live in three languages
- [ ] Hosting and licence confirmed for the Temelash entity
- [ ] SMS and Telegram providers on paid production accounts with spend alerts
- [ ] Accountant has confirmed receipt and tax requirements
- [ ] Support playbook and agent guide handed out
- [ ] Rollback plan for every release written and rehearsed

11. Open decisions and risks

Decide in Phase 0, with Robel and Truman.

Decision	Recommended default	Why
Backend stack	TypeScript, NestJS, PostgreSQL, Redis queue	Matches the team's other production work, strong typing for money and permissions. Supabase would be faster to start but phone-and-PIN login, Telegram `initData` checks and local SMS do not fit its auth well
Frontend	Next.js for all four apps, one shared component package	Customer app is already planned in Next.js; one toolchain for a small team
SMS provider	Pick one Ethiopian provider with delivery reports and a sender name; keep a second as fallback	Ethiopian numbers and the telecom's rules differ from global providers
Hosting	A region close to Addis Ababa with managed Postgres	Latency on weak networks; licence and hosting must be settled before Release 3
Telegram role	Customer app only; the bot sends stamp, reward and news messages	Staff and owners stay on the web apps
Online payments	Not in Release 1 or 2; add Telebirr or Chapa after reconciliation is stable	Manual payments are already designed and audited
Receipts	Keep the simple receipt; add tax details only if the accountant requires them	Avoids building a fiscal system by guess
Who may run the kill switch	Owner role only	Not enforced in the prototype; the safest default

Risks.

Risk	Effect	What to do
Fake or farmed stamps (staff or owners stamping themselves)	Rewards given for nothing; trust lost	Rotating QR, per-staff hourly limits, flags, approvals for typed stamps, owner and admin fix log
SMS cost and delivery in Ethiopia	Owners cannot sign in; costs climb	Provider fallback, cost report from Phase 6, app-first messaging
Weak networks in shops	Failed or duplicate stamps	Staff offline queue with server-side replay rules; load tests with throttling
Amharic and Oromo quality	Owners distrust the product	Native review in Phase 9, translators on the pilot, a word list for money and loyalty terms
Telegram mini-app limits	Features behave differently by phone and version	Test on three Android models and iOS early in Phase 2; keep a web fallback in the backlog
Money mistakes by hand entry	Wrong balances, commission disputes	Append-only rows, corrections not edits, month-end reconciliation, Finance role separation
Small team, long list	Slips and shortcuts	Hold the exit gates; ship the pilot at Phase 4 and resist adding scope before it
Licence and hosting not ready	Cannot go public	Start the licence process now; it gates Release 3, not the pilot

12. Working agreement and kickoff prompt

How the building Claude should work.

- Work one phase at a time. At the start of each phase, restate its goal, list the boards it will follow, and name anything unclear before writing code.

- Open the prototype boards for a screen before building it, and match wording, order of fields, error messages and empty states. If the prototype and this document disagree, the prototype wins for screens and this document wins for architecture; flag the difference.
- Write the acceptance tests of a phase first, then the code.
- Never invent a business rule. If a rule is missing, add it to section 11 as an open decision and ask Truman.
- Keep a `DECISIONS.md` with the date, the decision and who made it.
- End every working session with: what is done, what is next, what is blocked, and what needs a human.
- Do not start the next phase until the current phase's exit gate is met and Truman or Robel has confirmed it.

Who decides what. Truman owns product, pricing, wording and priorities. Robel owns architecture, security and the technical plan. A change to money rules, permissions or the data model needs both.

Kickoff prompt, ready to paste.

```
You are the lead engineer building Temelash Card, a digital stamp-card
loyalty product for businesses in Ethiopia. Two things are attached: (1) a
finished design prototype with 336 clickable boards covering the customer app
(Telegram Web App), the staff app, the owner and branch-manager app, and the
internal Admin back-office, in English, Amharic and Afaan Oromoo; (2) the
document "Temelash Card — Phased Build Plan".

Read the plan end to end. Then do only this:
1. Summarise the product, the four apps and the ten phases in your own words,
in under 300 words.
2. List every open decision in section 11 and state your recommended answer
for each, with one reason.
3. List anything in the prototype that the plan does not cover or that
contradicts it.
4. Propose the repo layout and the first week of Phase 0 as a task list.

Do not write application code until Truman confirms your answers to steps 2
and 3. From then on, follow section 12, "How the building Claude should
work". The prototype is the specification for screens, wording and flows. The
server decides permissions, money and stamps. Money and stamps are append-
only. Every action is audit-logged. Every string ships in English, Amharic
and Afaan Oromoo.
```